[Trung Nguyen]Toolchain & Cross-Compilation

Cross-Compiling Toolchain

1. Concepts

A **cross-compiling toolchain** is a set of tools that work in sequence (a *chain*). It runs on one platform (the Host) but produces binaries (executables, libraries, kernels…) for a **different** platform (the Target).

The two platforms may differ in:

- CPU architecture
- ABI (Application Binary Interface)
- Operating system
- C library

1.1. Build, Host and Target

Concept	Role	Example
Build	Machine where the toolchain itself is built	Vendor's x86_64 Linux server (ARM, Linaro)
Host	Machine where the toolchain runs and the developer compiles code	x86_64 laptop running Linux/Windows
Target	Machine that runs the binaries produced by the toolchain	Raspberry Pi (ARM), STM32 (Cortex-M), Wi-Fi router (MIPS)

Toolchain type	Relationship
Native toolchain	`build == host == target`
Cross-compiling toolchain	`build == host != target`

> Running a cross-compiled binary directly on the Host fails with `Exec format error` because the CPU instruction sets differ.

These concepts map to the `--build`, `--host` and `--target` options of an **Autoconf** `configure` script. By default, Autoconf guesses them from the current machine.

1.2. Why cross-compile?

- **Limited target resources:** microcontrollers (Arduino, ESP32, STM32) have only tens of KB of RAM and no storage or OS to run a compiler.
- **Build speed:** building the Linux kernel on a small ARM board can take many hours; on a multi-core PC it takes minutes.
- **Bootstrapping:** a new CPU architecture or OS has no compiler running on it yet, so existing machines must cross-compile the first one.

---

2. Naming Convention (Target Triplet)

A cross-toolchain is identified by the prefix of its commands:

```
<arch>-<vendor>-<os>-<abi>
```

Field	Meaning	Common values
`arch`	CPU architecture	`arm`, `aarch64`, `riscv64`, `mips`, `x86_64`
`vendor`	Vendor (may be omitted)	`none`, `unknown`, `pc`, `poky`, `buildroot`
`os`	Target operating system	`linux`, `none` (bare-metal), `w64`
`abi`	Calling convention, file format, floating-point handling	`gnu`, `gnueabihf` (hard-float), `eabi`, `musl`

Examples: `arm-none-eabi-gcc`, `arm-linux-gnueabihf-gcc`, `aarch64-linux-gnu-gcc`.

---

3. Bare-metal Toolchain vs. Linux Toolchain

The two main values of the `<os>` field:

- `none` – bare-metal: the program runs directly on the hardware (or on an RTOS) and initializes the CPU, peripherals and memory itself. *Freestanding* C environment.
- `linux` – the program runs on Linux and accesses hardware through the kernel via **system calls**. *Hosted* C environment.

3.1. Bare-metal toolchain (`os = none`)

- Used for development **without an operating system**.
- The C library is usually **newlib** (or newlib-nano):
  - Provides functions that need no OS (`strcpy`, `memcpy`, `sin`, `snprintf`...).
  - Functions that need I/O or memory (`printf`, `malloc`) rely on *stubs* such as `_write`, `_read`, `_sbrk` (libnosys) that the developer implements for the hardware (e.g. `_write` sends data to a UART).
- Can build **bootloaders** (U-Boot) and the **Linux kernel**, since they are freestanding programs built with `-nostdlib`.
- **Cannot** build Linux userspace code: it lacks a libc that speaks Linux system calls and the kernel headers.

3.2. Linux toolchain (`os = linux`)

- Used for development of software running on **Linux**; ships with Linux kernel headers.
- Linux-specific C libraries: **glibc**, **uClibc-ng**, **musl**.
- The libc turns C functions (`open`, `read`, `write`, `fork`, `malloc`) into Linux system calls.
- Builds **Linux userspace** code (applications, daemons, `.so` libraries), and can also build bootloaders/kernels with `-ffreestanding -nostdlib -nostartfiles`.

3.3. Detailed comparison

Criterion	Bare-metal	Linux
Example	`arm-none-eabi-gcc`	`arm-linux-gnueabihf-gcc`
C environment	Freestanding	Hosted
C library	newlib / newlib-nano	glibc, musl, uClibc-ng

Criterion	Bare-metal	Linux
System calls	User-written stubs (`_write`, `_sbrk`)	Linux syscalls (`write`, `brk`, `mmap`…)
File format	Linked as ELF, then `objcopy` to `.bin`/`.hex` for flashing	ELF runs directly; the kernel loads it into RAM
Entry point	Reset handler (startup file)	Kernel jumps to libc's `_start` → `main`
Memory layout	Custom linker script (`.ld`)	Default linker script
Virtual memory	None; physical addresses	Yes, via the MMU
Shared libraries	Usually none; static linking	`.so` supported; needs a full sysroot
Multitasking	Self-written or RTOS	Provided by the kernel
Hardware	MCU without MMU (Cortex-M)	CPU with MMU (Cortex-A, Raspberry Pi)
Builds bootloader/kernel	Yes	Yes (with libc disabled)
Builds Linux userspace	No	Yes

3.4. Consequences of using the wrong one

- Building a Raspberry Pi (Linux) program with `arm-none-eabi-gcc` → **does not run**; the program does not know how to make Linux system calls.
- Writing Cortex-M firmware with `arm-linux-gnueabihf-gcc` → **unusable**; glibc requires the Linux kernel and an MMU, which the MCU lacks.

> Both toolchain types can share the same compiler (GCC) and ARM ISA; they differ in **target OS** and **C library**.

---

4. Toolchain Components

Turning source code into an executable is a pipeline: each tool's output is the next tool's input.

```
.c → [cpp] → [cc1] → .s → [as] → .o → [ld] → ELF
```

4.1. Binutils

Binutils (*binary utilities*) is a set of low-level tools that create, link, convert and inspect binary files (`.o`, `.a`, `.so`, ELF, `.bin`, `.hex`) for the Target architecture.

The two core tools:

- **`as`** (assembler): turns assembly (`.s`, `.S`) into an object file (`.o`).
- **`ld`** (linker): links `.o` files and libraries (`.a`, `.so`), resolves addresses and lays out memory according to a linker script, producing an executable, a shared library or another object file.

Analysis and utility tools:

Tool	Purpose
`ar`	Packs `.o` files into a static library `.a`
`objcopy`	Converts formats (e.g. `.elf` → `.bin`/`.hex`)
`objdump`	Disassembles machine code, shows sections
`readelf`	Shows ELF structure (headers, symbol table, dependencies)
`strip`	Removes debug symbols to reduce size
`nm`	Lists symbols (functions, global variables) and their addresses
`size`	Reports `.text`, `.data`, `.bss` sizes

Because each architecture has its own instruction set, every Binutils command in a cross-toolchain carries the triplet prefix: `arm-linux-gnueabihf-as`, `arm-linux-gnueabihf-objdump`...

4.2. GCC (GNU Compiler Collection)

GCC translates high-level source code (C, C++, Fortran, Ada...) into Target assembly. The `gcc` command has two roles:

- **Compiler proper** (`cc1`, `cc1plus`): preprocessing, parsing, optimization, emitting `.s`.
- **Compiler driver**: automatically runs `cpp` → `cc1` → `as` → `ld` together with the standard libraries.

Bundled runtime libraries:

- **`libgcc`**: low-level routines the CPU does not support natively (integer division on older ARM, soft-float...).
- **`libstdc++`**: the C++ standard library.

4.3. Linux Kernel Headers

`.h` files extracted and *sanitized* from the kernel sources (`include/uapi/`, `include/asm/`...) that define the interface between userspace and the kernel. **Only present in Linux toolchains.**

Kernel headers provide:

- **System call numbers**: `__NR_open`, `__NR_read`, `__NR_write`...
- **Constants and macros**: `ioctl()` codes, `O_RDWR`, `SIGINT`...
- **Data structures** (`struct`) exchanged between userspace and the kernel.

Version rule: the Linux userspace ABI is backward compatible, so the kernel headers version in the toolchain must be ≤ the kernel version running on the Target.

> Example: toolchain built with 6.6 headers, board running kernel 5.10 → the libc may use new system calls that 5.10 lacks → runtime failures.

4.4. C Library (libc)

The libc is the bridge between userspace applications and the kernel. When the toolchain is built, the libc provides:

- **Standard headers** (`stdio.h`, `stdlib.h`, `unistd.h`...) in `sysroot/usr/include`.
- **Binary libraries** (`libc.so`, `libc.a`, `ld-linux.so`) in `sysroot/lib`, which must be copied to the Target root filesystem.

Two main jobs:

1. Implement the ISO C / POSIX functions: strings, math, memory, I/O, `pthread`, `socket`.
2. **System call wrappers**: turn `read()`, `write()`, `open()` into a special instruction (e.g. `svc` on ARM) that asks the kernel to do the work.

Linux C libraries compared:

	glibc	uClibc-ng	musl
Target use	Servers, desktops, larger embedded systems	Resource-constrained embedded, including MMU-less (uClinux)	Small embedded systems (Alpine, OpenWrt)
Pros	Full POSIX/GNU compliance, high performance, broad compatibility	Highly configurable (menuconfig), small	Very small (a few hundred KB), good static linking, low RAM use

	glibc	uClibc-ng	musl
Cons	Large (several MB), poor static linking	No stable ABI; software written for glibc may need patches	Lacks some GNU extensions

For bare-metal, the libc is usually **newlib** (see section 3.1).

4.5. Debugger

`gdb` follows a **gdbserver** model: gdbserver runs on the Target, **cross-gdb** runs on the Host, connected over the network or JTAG/SWD.

---

5. GCC Dependencies and Build Order

5.1. Host math libraries

GCC itself (running on the Host) needs these libraries to evaluate constants and optimize at compile time:

Library	Required	Role
GMP	Yes	Arbitrary-precision arithmetic (integers, rationals, floats)
MPFR	Yes	Multiple-precision floating point with correct rounding (built on GMP)
MPC	Yes	High-precision complex numbers (built on GMP, MPFR)
ISL	No	Graphite loop optimizations

> These libraries are built for the **Host** architecture and are never copied to the Target.

5.2. Circular dependency: GCC ↔ C library

- GCC needs **Binutils** (`as`, `ld`) to produce machine code.
- Building the **libc** requires GCC, but building a **full GCC** (libgcc, pthreads, libstdc++) requires the libc and kernel headers.

Solution – two-stage bootstrap:

1. Build **Binutils**.
2. Install the **Linux kernel headers**.
3. Build a **stage 1 GCC** (minimal: C only, no threads, no standard library).
4. Use the stage 1 GCC to build the **C library**.
5. Build the **stage 2 GCC** (full C/C++).

---

6. Sysroot

The **sysroot** is a directory on the Host that acts as the Target's emulated root (`/`) during compiling and linking. It contains the headers and libraries built for the Target.

6.1. Why a sysroot?

If a cross-compiler searched the Host's `/usr/include` and `/usr/lib`, it would pick up x86_64 files and mix them into ARM code (**host pollution**), causing link failures. With a sysroot, the compiler prefixes its default search paths:

- `/usr/include` → `<sysroot>/usr/include`
- `/lib`, `/usr/lib` → `<sysroot>/lib`, `<sysroot>/usr/lib`

6.2. Layout

Directory	Contents
`/lib`	`libc.so`, `libm.so`, `libpthread.so`, `libgcc_s.so`, `ld-linux-*.so`
`/usr/include`	libc headers, kernel headers (`linux/`, `asm/`), third-party headers
`/usr/lib`	Static libraries `.a`, `crt1.o`, `crti.o`, `crtn.o`, other `.so` libraries

6.3. Usage

Show the default sysroot:

```
arm-linux-gnueabihf-gcc --print-sysroot
```

Use a custom sysroot (e.g. a Buildroot/Yocto staging directory containing OpenSSL, Qt…):

```
arm-linux-gnueabihf-gcc --sysroot=/path/to/target-sysroot main.c -o app
```

6.4. Sysroot vs. rootfs

Criterion	Sysroot	Rootfs
Location	On the Host (build time)	On the Target (run time)
Contents	Headers, `.a`, `.so` with debug symbols	Only stripped `.so`, executables (`/bin`, `/sbin`), configuration (`/etc`)

---

7. Multilib Toolchain

A **multilib toolchain** is a single toolchain that ships several prebuilt copies of its libraries (libc, libgcc, libstdc++) for different hardware/ABI variants of the same CPU family.

7.1. Why multilib?

- GCC can target many variants with flags alone (`-mcpu=cortex-m4`, `-mfloat-abi=hard`, `-m32` …).
- But the libc must be built with the **same** flag combination. E.g. code built with `-mfloat-abi=hard` against a soft-float-only libc → `ld` reports an ABI mismatch.
- Multilib solves this by storing several subdirectories in the sysroot, one per flag combination, instead of installing separate toolchains.

7.2. How it works

Given `-march`, `-mcpu`, `-mabi`, `-mfloat-abi`, the `gcc` driver looks up its multilib mapping table and points `ld` to the matching library directory.

7.3. Examples

Bare-metal (`arm-none-eabi-gcc`), shared across Cortex-M0 → M7 and Cortex-R:

- `thumb/v6-m/nofp` – Cortex-M0/M0+
- `thumb/v7-m/nofp` – Cortex-M3
- `thumb/v7e-m/fpv4-sp/hard` – Cortex-M4F

Linux (`x86_64-linux-gnu-gcc`): `-m64` (default) links against `/lib64`, `-m32` against `/lib32`.

7.4. Checking

```
<prefix>-gcc -print-multi-lib
```

- Prints only `.;` → single-lib toolchain.
- Prints several lines like `dir;@flag1@flag2` → multilib toolchain.

Show the directory selected for a given set of flags:

```
arm-none-eabi-gcc -mcpu=cortex-m4 -mfloat-abi=hard -mfpu=fpv4-sp-d16 -print-multi-directory
```

---

8. Toolchain vs. SDK

```
SDK = Toolchain + Extended sysroot (Target libraries) + Host tools
```

Criterion	Toolchain	SDK
Produced by	Crosstool-NG, prebuilt (ARM, Linaro, Bootlin)	Buildroot (`make sdk`), Yocto (`bitbake <image> -c populate_sdk`)
Sysroot	libc, libstdc++, kernel headers only	Every library installed in the Target image, with `.pc` files for `pkg-config`
Tied to an image	No, generic	Yes, matches the exact library versions on the board
Host tools	Compiler, binutils, gdb	Plus cmake, meson, pkg-config, Qt tools, QEMU…
Environment setup	Set `PATH` / `CROSS_COMPILE` manually	`source environment-setup-*` sets `CC`, `CFLAGS`, `LDFLAGS`, `PKG_CONFIG_*`
Best for	Bootloader, kernel, libc-only apps; base for Buildroot/Yocto	Application development for a specific product

Key points:

- A plain toolchain **cannot** build apps that depend on third-party libraries (OpenSSL, Qt, ALSA…), since their headers and libraries are not in its sysroot.
- In an SDK, `$CC` already includes `--sysroot` and CPU flags, so Makefiles should **append** (`CFLAGS +=`) rather than overwrite `CC`/`CFLAGS`/`LDFLAGS`.

Revision #1
Created 2026-10-21 07:16:51 UTC by trungnguyen
Updated 2026-10-21 07:18:52 UTC by trungnguyen